第1章 SLAM问题表述及通用框架

1.1 SLAM问题表述

直观表述：机器人在未知环境中从任一位置开始移动，移动过程中根据对环境的探测确定自身位姿与运动轨迹，同时构建出室内地图。

数学表述：知道运动测量的读数$\boldsymbol{u}$，以及传感器的读数$\boldsymbol{z}$，根据运动方程和观测方程，求解定位问题（估计$\boldsymbol{x}$）和建图问题（估计$\boldsymbol{y}$）。

（1）位姿--轨迹：机器人各时刻的位姿如下，一组位姿构成机器人一段时间内的运动轨迹。

$$\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_k$$

（2）路标--地图：地图由多个路标点组成。机器人行驶过程中，传感器不断探测路标点，得到观测数据。

$$\boldsymbol{y}_1, \boldsymbol{y}_2, \cdots, \boldsymbol{y}_k$$

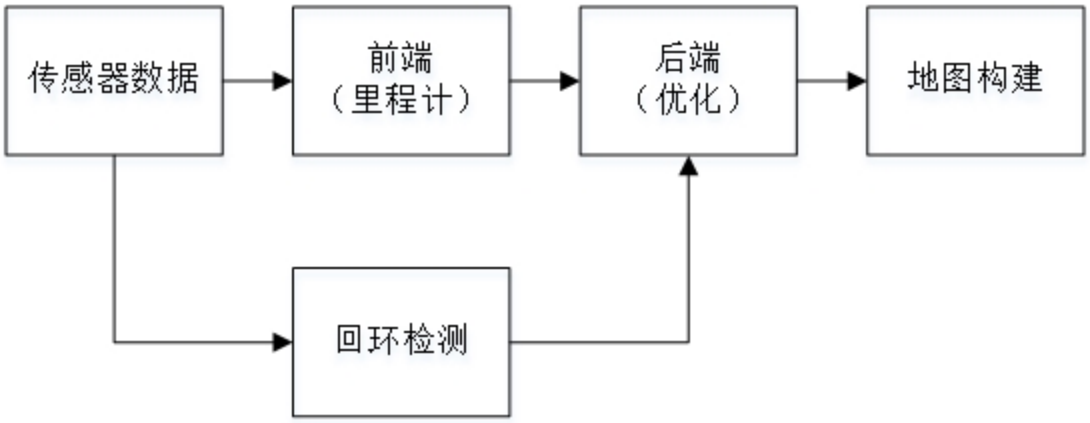
（3）运动方程：由上一时刻的位姿得到当前时刻的位姿，$\boldsymbol{u}_k$为输入，$\boldsymbol{\omega}_k$是运动传感器的噪声 。

$$\boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{\omega}_k\right)$$

（4）观测方程：机器人在位姿点$\boldsymbol{x}_k$看到某个路标点$\boldsymbol{y}_j$，产生一个观测数据$\boldsymbol{z}_{k,j}$。$\boldsymbol{v}_{k,j}$是观测噪声 。

$$\boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}\right)$$

1.2 通用框架



1.2.1 传感器

传感器类型：激光雷达，相机（单目/双目/RGB-D），码盘，IMU等。

数据读取和处理：稳定的系统中会使用多个传感器，因此涉及到数据读取、同步、外点剔除等问题。

1.2.2 前端

前端主要实现里程计，里程计用于估计机器人的运动轨迹，从而解决SLAM中的定位问题。

- **基础方案：**
    - 视觉里程计：利用相邻帧间的图像估计相机运动，并恢复场景的空间结构。将相邻时刻的运动串起来，就构成了轨迹。
    - 码盘里程计：基于机器人电机编码器实现。如果机器人行驶过程中，轮子打滑会造成里程计误差很大。
- **融合方案：**
    - 单个传感器实现的里程计噪声较大，存在累计误差，可用多传感数据进行融合得到里程计信息，常见有相机、码盘、IMU、RTK组合融合方案。

1.2.3 后端

后端用于优化，从带有噪声的数据中，估计整个系统的状态（定位和地图），以及这个状态的估计的不确定性多大，即最大后验概率估计（Maximum-a-Posteriori，MAP）。优化方法有：

滤波器、非线性优化、图优化。

1.2.4 回环检测

漂移问题：机器人经过一段时间回到了原点，但由于漂移，位置估计并没有回到原点。

回环检测：通过判断环境相似性，检测机器人是否回到原点，回环检测结果发送给后端，后端将轨迹和地图调整成回环检测的结果。因此如果回环检测成功，可显著减小累计误差。

视觉回环检测：计算图像数据相似性的算法。

1.2.5 地图构建

- **度量地图：**强调精确地表示地图中物体的位置关系，分为稀疏地图和稠密地图，对于定位，稀疏地图就足够了。而用于导航时，往往需要稠密的地图。
    - 稀疏地图：进行了一定程度的抽象，并不需要表达所有的物体。
    - 稠密地图：着重于建模所有看到的东西。稠密地图按照某种分辨率，由许多个小块组成。二维度量地图是许多个小格子（Grid），三维则是许多小方块（Voxel）。一个小块含有占据、空闲、未知三种状态，以表达该格内是否有物体。当导航算法查询某个空间位置时，地图能够给出该位置是否可以通过的信息。
- **拓扑地图：**拓扑地图是一个图，由节点和边组成，只考虑节点间的连通性，放松了对精确位置的需要，去掉了地图的细节问题。

第2章 刚体运动与位姿表示

2.1 位姿表示

位置：如果在三维空间中，使用三个空间坐标表示：

$$\boldsymbol{x} = [x, y, z]$$

姿态：2D姿态用一个旋转角θ表示。3D姿态可用旋转矩阵、变换矩阵、欧拉角、四元数等表示。

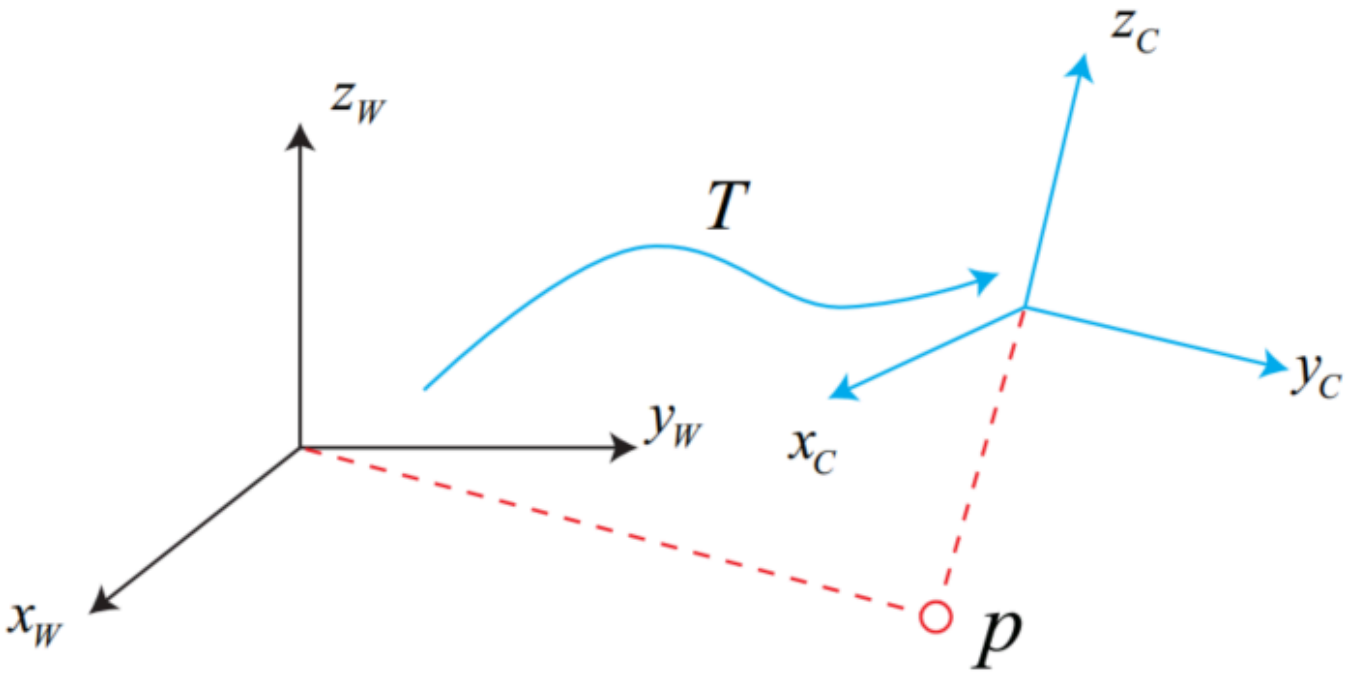
2.2 旋转矩阵与变换矩阵

2.2.1 坐标系间的欧式变换

坐标系间的欧式变换：同一个向量在各个坐标系下的长度和夹角都不会发生变化，如相机运动。一个欧式变换由平移和旋转组成。

某个向量$\boldsymbol{p}$在世界坐标系和相机坐标系中的坐标分别为$\boldsymbol{p}_w$和$\boldsymbol{p}_c$，通常在计算中需要转换在同一个坐标系中进行，二者存在转换关系用矩阵T表示。

- **世界坐标系：**机器人运动过程中，世界坐标系固定不动，下图左侧坐标系。
- **相机坐标系：**相机本身存在一个坐标系，随相机的运动而变化，下图右侧坐标系。



2.2.2 旋转矩阵

2D空间下对旋转矩阵的推导较为直观，由2D空间可引出3D空间下旋转矩阵的表示。

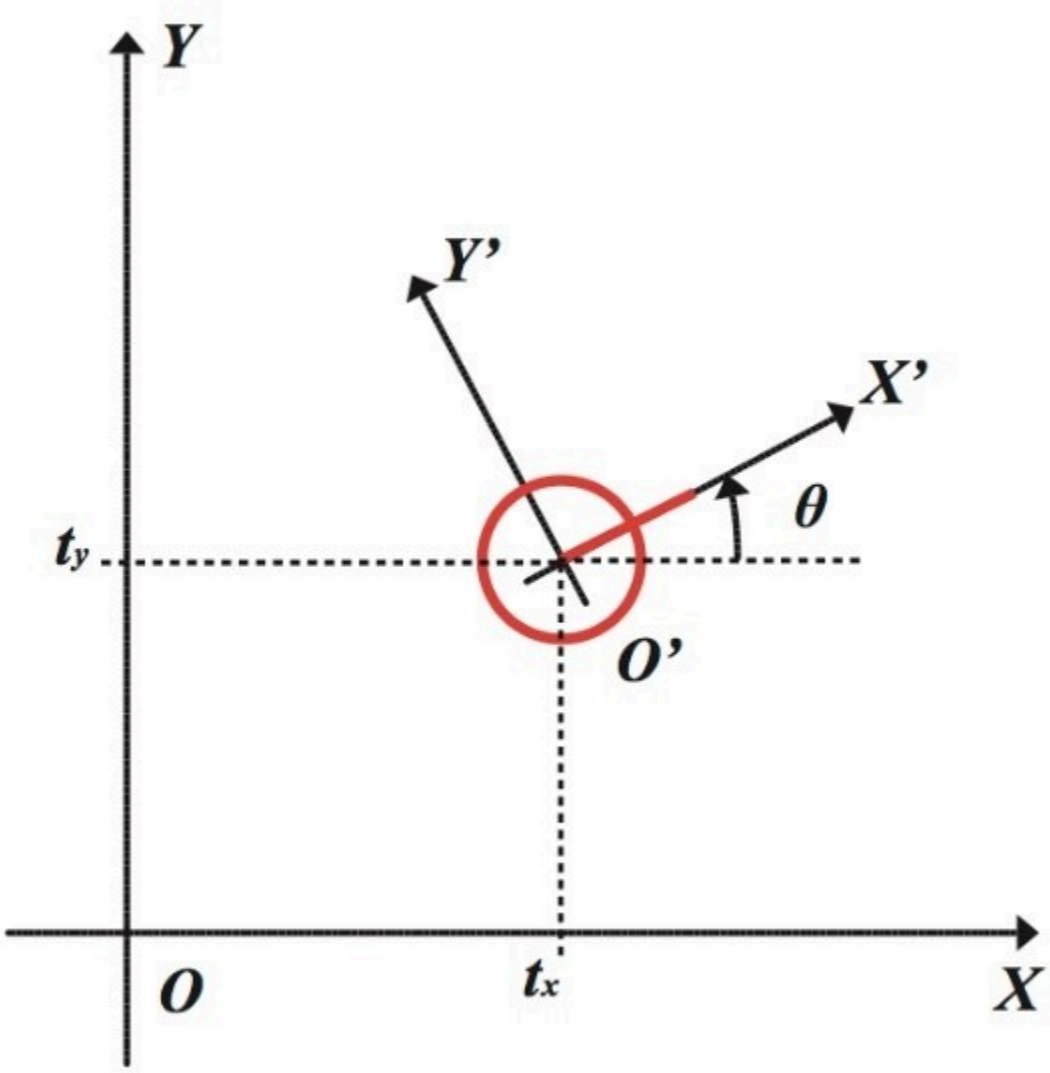
2.2.2.1 2D空间下

设某点在世界坐标系中的坐标下标为w，对应在机器人坐标系中的坐标下标为r，根据下图所示，可推导得到二者存在如下关系：

$$\begin{cases} \\ x_w = x_r \cos\theta - y_r \sin\theta + t_x \\ y_w = x_r \sin\theta + y_r \cos\theta + t_y \end{cases}$$

将上述关系改写成矩阵形式，其中$\boldsymbol{R}$为旋转矩阵，$\boldsymbol{t}$为平移矢量。

$$\begin{aligned} \boldsymbol{x}_w &= \boldsymbol{R}\boldsymbol{x}_r + \boldsymbol{t} \\ &= \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \boldsymbol{x}_r + \begin{bmatrix} lt_x \\ t_y \end{bmatrix} \end{aligned}$$



2.2.2.2 3D空间下

变换前后坐标系的基分别为(e_1, e_2, e_3)和$(e_{1'}, e_{2'}, e_{3'})$，则同一个向量在两个坐标系下的坐标分别为$[a_1, a_2, a_3]_T$和$[a_{1'}, a_{2'}, a_{3'}]_T$。根据坐标定义，有：

$$\begin{bmatrix} & & \\ e_1, & e_2, & e_3 \\ & & \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} & & \\ e_{1'}, & e_{2'}, & e_{3'} \\ & & \end{bmatrix} \begin{bmatrix} a_{1'} \\ a_{2'}^1 \\ a_{3'}^2 \end{bmatrix}$$

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} e_T e_{1'} & e_T e_{2'} & e_T e_{3'} \\ e_T^1 e_{1'}^1 & e_T^1 e_{2'}^2 & e_T^1 e_{3'}^3 \\ e_{3T}^2 e_{1'}^1 & e_{3T}^2 e_{2'}^2 & e_{3T}^2 e_{3'}^3 \end{bmatrix} \begin{bmatrix} a_{1'} \\ a_{2'}^1 \\ a_{3'}^2 \end{bmatrix} \triangleq \boldsymbol{R} \boldsymbol{a}_{1'}$$

旋转矩阵是行列式为1的正交矩阵（正交：$\boldsymbol{R}\boldsymbol{R}_T = \boldsymbol{E}$），它的逆或转置代表了相反的旋转，定义旋转矩阵的集合（特殊正交群）。

$$\boldsymbol{a}_{1'} = \boldsymbol{R}_{-1} a = \boldsymbol{R}_T \boldsymbol{a}$$

$$SO(n) = \{\boldsymbol{R} \in R_{n \times n} | \boldsymbol{R}\boldsymbol{R}_T = \boldsymbol{I}, \det(\boldsymbol{R}) = 1\}$$

同2D空间，使用旋转矩阵和平移向量表示变换如下（转置表示相反的旋转）。

$$\boldsymbol{a}_{1'} = \boldsymbol{R}\boldsymbol{a} + \boldsymbol{t}$$

2.2.3 变换矩阵和齐次坐标

- **变换矩阵：**利用旋转矩阵表示，如果进行多次变换，会进行多次计算，显得很复杂。引入变换矩阵和齐次坐标，$\boldsymbol{T}$即为变换矩阵。

$$\begin{bmatrix} \boldsymbol{a}_{1'} \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}_T & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix} \triangleq \boldsymbol{T} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix}$$

两次变换可表示为（将向量默认记作齐次坐标方式）：

$$\boldsymbol{b}=\boldsymbol{T}_1\boldsymbol{a},\quad \boldsymbol{c}=\boldsymbol{T}_2\boldsymbol{b}=\boldsymbol{T}_2\boldsymbol{T}_1\boldsymbol{a}$$

定义变换矩阵的集合（特殊欧式群）：

$$SE(3)=\{\boldsymbol{T}\in R_{4\times4}|\boldsymbol{R}\in SO(3),\boldsymbol{t}\in R_3\}$$

- **齐次坐标：**上述在一个三维向量的末尾加1，变为四维向量，称为齐次坐标，是一种运算技巧。

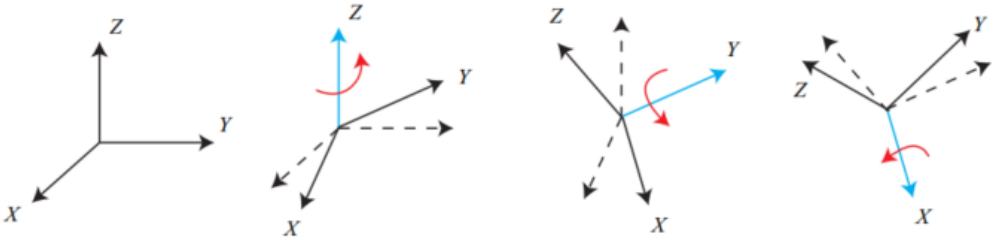
2.3 旋转向量

任意旋转都可用一个旋转轴和一个旋转角来表示。因此可以使用一个向量（旋转向量），其方向与旋转轴一致，而长度等于旋转角。即：

- 旋转轴：$\boldsymbol{n}$
- 旋转角：θ
- 旋转向量：$\theta\boldsymbol{n}$

2.4 欧拉角

- **欧拉角的提出：**旋转向量并不直观，不清楚具体在朝哪个方向旋转。欧拉角使用了3个分离的转角，把一个旋转分解成3次绕不同轴的旋转。分解方式有许多种，即绕着X、Y、Z三轴旋转的顺序不同，如先绕X轴旋转，再绕Y轴，最后绕Z轴，就得到了一个XYZ轴的旋转。
- **示例：**最常用的欧拉角表达方式：把旋转分解成沿三个轴转动的量：偏航角-俯仰角-滚转角（$yaw-pitch-roll$）。三次旋转如下：
    - 绕物体的Z轴旋转，得到偏航角yaw；
    - 绕旋转之后的Y轴旋转，得到俯仰角$pitch$；
    - 绕旋转之后的X轴旋转，得到滚转角$roll$。



- **缺点：**会碰到著名的万向锁问题：在俯仰为±90°时，第一次旋转与第三次旋转将使用同一个轴，使得系统丢失了一个自由度（由三次旋转变成了两次旋转），这

被称为奇异性问题。此外，它也不易于插值和迭代。因此它在SLAM中用处有限。不过想验证自己算法输出的姿态是否有错时，转换成欧拉角可快速辨认结果是否有错。

2.5 四元数

2.5.1 定义

- **四元数：**旋转矩阵用9个量描述3自由度的旋转，具有冗余性；欧拉角和旋转向量是紧凑的，但具有奇异性。四元数是一种扩展的复数，既是紧凑的，又没有奇异性。一个四元数拥有1个实部和3个虚部。

$$\boldsymbol{q} = q_0 + q_1 i + q_2 j + q_3 k$$

$$\begin{cases} i_2 = j_2 = k_2 = -1 \\ ij = k, ji = -k \\ jk = i, kj = -i \\ ki = j, ik = -j \end{cases}$$

也可用1个标量和1个向量表示四元数如下，其中$\boldsymbol{s}$为实部，$\boldsymbol{v}$为虚部。实四元数：虚部为0；虚四元数：实部为0。

$$\boldsymbol{q} = [\boldsymbol{s}, \boldsymbol{v}], \quad \boldsymbol{s} = q_0 \in R, \quad \boldsymbol{v} = [q_1, q_2, q_3]_T \in R_3$$

- **四元数表示旋转：**设空间中一个三维点$\boldsymbol{p} = [0, x, y, z] = [0, \boldsymbol{v}]$（以虚四元数形式表示），以及一个由单位向量$\boldsymbol{n} = [n_x, n_y, n_z]_T$和$\theta$确定的旋转。用四元数形式表示该旋转如下：

$$\begin{aligned} \boldsymbol{q} &= \left[\cos \frac{\theta}{2}, \boldsymbol{n} \sin \frac{\theta}{2} \right] \\ &= \left[\cos \frac{\theta}{2}, n_x \sin \frac{\theta}{2}, n_y \sin \frac{\theta}{2}, n_z \sin \frac{\theta}{2} \right]_T \end{aligned}$$

旋转后的点表示如下，该计算结果的实部为0，虚部的3个分量表示旋转后3D点的坐标。

$$p_{'} = qpq_{-1}$$

2.5.2 四元数运算

$$q_a = [s_a, v_a] = s_a + x_a i + y_a j + z_a k$$

$$q_b = [s_b, v_b] = s_b + x_b i + y_b j + z_b k$$

- 加法和减法

$$q_a \pm q_b = [s_a \pm s_b, v_a \pm v_b]$$

- 乘法

$$\begin{aligned} q_a q_b =& s_a s_b - x_a x_b - y_a y_b - z_a z_b \\ &+ (s_a x_b + x_a s_b + y_a z_b - z_a y_b) i \\ &+ (s_a y_b - x_a z_b + y_a s_b + z_a x_b) j \\ &+ (s_a z_b + x_a y_b - y_b x_a + z_a s_b) k \end{aligned}$$

$$q_a q_b = \left[s_a s_b - v_{\underset{a}{T}} v_b, s_a v_b + s_b v_a + v_a \times v_b\right]$$

- 共轭

$$q_{\underset{a}{*}} = s_a - x_a i - y_a j - z_a k = [s_a, -v_a]$$

$$q_* q = q q_* = \left[_{s_{\underset{a}{2}} + v_{\underset{a}{T}} v}, 0\right]$$

- 模长

$$\|q_a\| = \sqrt{s_{\underset{a}{2}} + x_{\underset{a}{2}} + y_{\underset{a}{2}} + z_{\underset{a}{2}}}$$

$$\|q_a q_b\| = \|q_a\| \, q_b\|$$

- 逆

$$\boldsymbol{q}_{-1}=\boldsymbol{q}_{*}/\|\boldsymbol{q}\|_{2}$$

$$\boldsymbol{q}\boldsymbol{q}_{-1}=\boldsymbol{q}_{-1}\boldsymbol{q}=\mathbf{1}$$

$$(\boldsymbol{q}_{a}\boldsymbol{q}_{b})_{-1}=\underset{b}{\boldsymbol{q}_{-1}}\underset{a}{\boldsymbol{q}_{-1}}$$

- 数乘与点乘

$$k\boldsymbol{q}=[ks,k\boldsymbol{v}]$$

$$\boldsymbol{q}_{a}\cdot\boldsymbol{q}_{b}=s_{a}s_{b}+x_{a}x_{b}+y_{a}y_{b}+z_{a}z_{b}$$

2.6 三者相互转换

2.6.1 四元数与旋转向量

四元数与旋转向量之间的相互转换：

$$\begin{cases} & \\ \theta & =2\arccos q_{0} \\ [n_{x},n_{y},n_{z}]_{T} & =[q_{1},q_{2},q_{3}]_{T}/\sin\frac{\theta}{2} \end{cases}$$

2.6.2 旋转向量与旋转矩阵

- **旋转向量到旋转矩阵：**由下式（罗德里格斯公式）表示，其中^表示向量的反对称：

$$\boldsymbol{R}=\cos\theta\boldsymbol{I}+(1-\cos\theta)\boldsymbol{n}\boldsymbol{n}_{\boldsymbol{T}}+\sin\theta\boldsymbol{n}_{\wedge}$$

$$\begin{aligned}\operatorname{tr}(\boldsymbol{R}) &=\cos\theta\operatorname{tr}(\boldsymbol{I})+(1-\cos\theta)\operatorname{tr}\left(\boldsymbol{n}\boldsymbol{n}_{T}\right)+\sin\theta\operatorname{tr}\left(\boldsymbol{n}_{\wedge}\right) \\ &=3\cos\theta+(1-\cos\theta) \\ &=1+2\cos\theta\end{aligned}$$

- **旋转矩阵到旋转向量：**（1）旋转角：通过上述第2式计算。

（2）旋转轴：n是$\boldsymbol{R}$特征值为1对应的特征向量。求解下述方程（旋转轴上的向量在旋转后不发生变化），再归一化：

$$\boldsymbol{R}\boldsymbol{n}=\boldsymbol{n}$$

2.6.3 四元数与旋转矩阵

基本思路：四元数转为旋转向量；再转换为旋转矩阵。

- 四元数到旋转矩阵：

$$\boldsymbol{R}=\begin{bmatrix}1-2q_2-2q_2 & 2q_1q_2+2q_0q_3 & 2q_1q_3-2q_0q_2\\ 2q_1q_2-2q_0q_3 & 1-2q_2-2q_2 & 2q_2q_3+2q_0q_1\\ 2q_1q_3+2q_0q_2 & 2q_2q_3-2q_0q_1 & 1-2q_2-2q_2\end{bmatrix}$$

- **旋转矩阵到四元数：**设旋转矩阵为$\boldsymbol{R}=m_{ij},ij\in[1,2,3]$。一个旋转矩阵对应的四元数并不唯一，因为$q$和$-q$可表示同一旋转。

$$\begin{cases}q_0=\\ q_1=\frac{\sqrt{\mathrm{tr}(\boldsymbol{R})+1}}{4q_0}\\ q_2=\frac{m_{31}-m_{13}}{4q_0}\\ q_3=\frac{m_{12}-m_{21}}{4q_0}\end{cases}$$

第3章 Eigen库 | 实现位姿表示

3.1 Eigen库

官网：http://eigen.tuxfamily.org。

文档：http://eigen.tuxfamily.org/dox/。

快速参考：http://eigen.tuxfamily.org/dox/group__QuickRefPage.html。

3.2 Eigen基本操作

完整代码：https://github.com/deng-ke/slam-base/tree/master/EigenMatrix。

3.2.1 模块和头文件

序号	模块	头文件	内容
1	Core	#include<Eigen/Core>	Matrix/Array类、基础运算
2	Geometry	#include<Eigen/Geometry>	旋转/平移/缩放，2D/3D变换
3	LU	#include<Eigen/LU>	求逆，行列式，LU分解
4	Cholesky	#include<Eigen/Cholesky>	LLT和LDLT Cholesky分解
5	Householder	#include<Eigen/Householder>	Householder变换
6	SVD	#include<Eigen/SVD>	SVD分解
7	QR	#include<Eigen/QR>	QR分解
8	Eigenvalues	#include<Eigen/Eigenvalues>	特征值/特征向量分解
9	Sparse	#include<Eigen/Sparse>	稀疏矩阵的存储和运算
10	Dense	#include<Eigen/Dense>	包含了1-4、6-8头文件
11	Eigen	#include<Eigen/Eigen>	Dense/Sparse头文件

3.2.2 Matrix

3.2.2.1 矩阵、向量定义

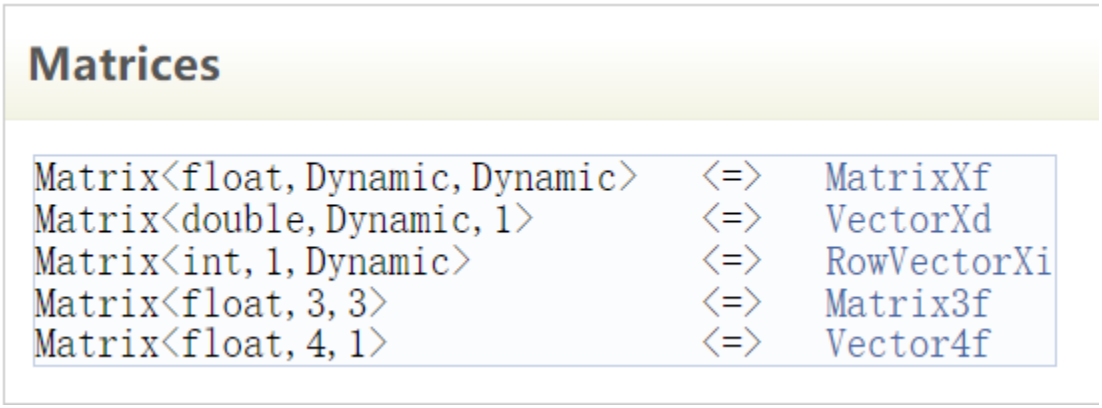
- **基本定义：**矩阵、向量都是Matrix模板类的对象，主要使用前三个参数，剩下为默认值。

```
Eigen::Matrix<float, 2, 3> matrix_23;
Eigen::Matrix<float,3,1> vd_3d;
```

- 动态大小：

```
Eigen::Matrix< double, Eigen::Dynamic, Eigen::Dynamic > matrix_dynamic1;
Eigen::Matrix< double, 1, Eigen::Dynamic > matrix_dynamic2;
```

- **简写定义：**typedef了一些简写：




```
Eigen::Vector3d v_3d;
Eigen::MatrixXd matrix_x;
```

3.2.2.2 矩阵、向量初始化

- 直接初始化：

```
matrix_23 << 1, 2, 3, 4, 5, 6;
```

- 初始化为**0**：

```
Eigen::Matrix3d matrix_33 = Eigen::Matrix3d::Zero();
```

```
matrix_23初始化：
1 2 3
4 5 6
matrix_33初始化为0：
0 0 0
0 0 0
0 0 0
matrix_33初始化为随机数：
 0.680375   0.59688 -0.329554
-0.211234  0.823295  0.536459
 0.566198 -0.604897 -0.444451
```

3.2.2.3 访问矩阵中的元素

```
for (int i=0; i<2; i++) {
    for (int j=0; j<3; j++)
        cout<<matrix_23(i,j)<<"\t";
    cout<<endl;
}
```

```
访问matrix_23中的元素：
1       2       3
4       5       6
```

3.2.3运算

- **四则运算：**以矩阵和向量相乘为例：

```
v_3d << 3, 2, 1;
vd_3d << 4,5,6;
Eigen::Matrix<float, 2, 1> result1 = matrix_23 * vd_3d;
cout << "matrix_23 * vd_3d = " << endl << result1 << endl;
Eigen::Matrix<double, 2, 1> result2 = matrix_23.cast<double>() * v_3d;
cout << "matrix_23 * v_3d = " << endl << result2 << endl;  //需显式转换类型
```

```
matrix_23 * vd_3d =
32
77
matrix_23 * v_3d =
10
28
```

- **其它运算：**包括转置、逆、迹等：

```
cout << "转置: " << endl << matrix_33.transpose() << endl;
cout << "各元素和: " << endl << matrix_33.sum() << endl;
cout << "迹: " << endl << matrix_33.trace() << endl;
cout << "数乘: " << endl << 10*matrix_33 << endl;
cout << "逆: " << endl << matrix_33.inverse() << endl;
cout << "行列式: " << endl << matrix_33.determinant() << endl;
//实对称矩阵可保证对角化成功:
Eigen::SelfAdjointEigenSolver<Eigen::Matrix3d> eigen_solver ( matrix_33.transpose()*matrix
cout << "特征值和特征向量: " << endl;
cout << "Eigen values = \n" << eigen_solver.eigenvalues() << endl;
cout << "Eigen vectors = \n" << eigen_solver.eigenvectors() << endl;
```

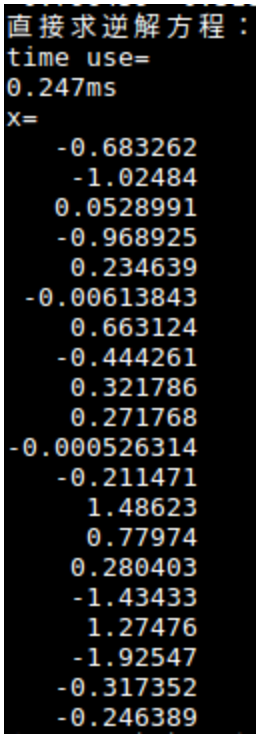
```
转置：
 0.680375 -0.211234  0.566198
  0.59688  0.823295 -0.604897
-0.329554  0.536459 -0.444451
各元素和：
1.61307
迹：
1.05922
数乘：
 6.80375   5.9688 -3.29554
-2.11234  8.23295  5.36459
 5.66198 -6.04897 -4.44451
逆：
-0.198521   2.22739    2.8357
  1.00605 -0.555135  -1.41603
 -1.62213   3.59308   3.28973
行列式：
0.208598
特征值和特征向量：
Eigen values =
0.0242899
 0.992154
  1.80558
Eigen vectors =
-0.549013 -0.735943  0.396198
 0.253452 -0.598296 -0.760134
-0.796459  0.316906 -0.514998
```

3.2.4 解方程

两种方法：直接求逆、利用QR分解。方法2更快。

- 直接求逆解方程：

```
clock_t time_stt = clock();
Eigen::Matrix<double,MATRIX_SIZE,1> x = matrix_NN.inverse()*v_Nd;
cout << "直接求逆解方程: " << endl;
cout << "time use=" << endl << 1000* (clock() - time_stt)/(double)CLOCKS_PER_SEC << "ms"<<
cout << "x=" << endl << x << endl;
```



- 用QR分解解方程：

```
time_stt = clock();
x = matrix_NN.colPivHouseholderQr().solve(v_Nd);
cout << "利用QR分解解方程: " << endl;
cout << "time use=" << endl << 1000*  (clock() - time_stt)/(double)CLOCKS_PER_SEC <<"ms" <
cout << "x=" << endl << x << endl;
```

```
利用QR分解解方程：
time use=
0.025ms
x=
   -0.683262
    -1.02484
   0.0528991
   -0.968925
    0.234639
 -0.00613843
    0.663124
   -0.444261
    0.321786
    0.271768
-0.000526314
   -0.211471
     1.48623
     0.77974
    0.280403
    -1.43433
     1.27476
    -1.92547
   -0.317352
   -0.246389
```

3.3 Eigen实现位姿表示

完整代码：https://github.com/deng-ke/slam-base/tree/master/EigenGeometry。

主要用到Geometry模块，该模块实现的类，包含旋转矩阵、变换矩阵、旋转向量、欧拉角、四元数等对位姿的描述方式。

class	**Eigen::AlignedBox< _Scalar, _AmbientDim >** An axis aligned box. More...
class	**Eigen::AngleAxis< _Scalar >** Represents a 3D rotation as a rotation angle around an arbitrary 3D axis. More...
class	**Eigen::Homogeneous< MatrixType, _Direction >** Expression of one (or a set of) homogeneous vector(s) More...
class	**Eigen::Hyperplane< _Scalar, _AmbientDim, _Options >** A hyperplane. More...
class	**Eigen::Map< const Quaternion< _Scalar >, _Options >** **Quaternion** expression mapping a constant memory buffer. More...
class	**Eigen::Map< Quaternion< _Scalar >, _Options >** Expression of a quaternion from a memory buffer. More...
class	**Eigen::ParametrizedLine< _Scalar, _AmbientDim, _Options >** A parametrized line. More...
class	**Eigen::Quaternion< _Scalar, _Options >** The quaternion class used to represent 3D orientations and rotations. More...
class	**Eigen::QuaternionBase< Derived >** Base class for quaternion expressions. More...
class	**Eigen::Rotation2D< _Scalar >** Represents a rotation/orientation in a 2 dimensional space. More...
class	**Scaling** Represents a generic uniform scaling transformation. More...
class	**Eigen::Transform< _Scalar, _Dim, _Mode, _Options >** Represents an homogeneous transformation in a N dimensional space. More...
class	**Eigen::Translation< _Scalar, _Dim >** Represents a translation transformation. More...

3.3.1 旋转向量/矩阵

AngleAxis定义旋转向量；Matrix3定义旋转矩阵。两种方式可将旋转向量转换为旋转矩阵。

```cpp
    /*****************************************************************************
     *旋转向量/矩阵；旋转向量转换为旋转矩阵
     *****************************************************************************/
    Eigen::AngleAxisd rotation_vector ( M_PI/4, Eigen::Vector3d ( 0,0,1 ) );
    Eigen::Matrix3d rotation_matrix = Eigen::Matrix3d::Identity();
    Eigen::Vector3d v ( 1,0,0 );
    cout.precision(3);
    /*两种方式：旋转向量 to 旋转矩阵*/
    cout<<"rotation matrix =\n"<<rotation_vector.matrix() <<endl;
    rotation_matrix = rotation_vector.toRotationMatrix();
    /*使用旋转向量变换*/
    Eigen::Vector3d v_rotated = rotation_vector * v;
    cout<<"(1,0,0) after rotation = "<<v_rotated.transpose()<<endl;
    /*使用旋转矩阵变换*/
    v_rotated = rotation_matrix * v;
    cout<<"(1,0,0) after rotation = "<<v_rotated.transpose()<<endl<<endl;
```

```
rotation matrix =
 0.707 -0.707      0
 0.707  0.707      0
     0      0      1
(1,0,0) after rotation = 0.707 0.707     0
(1,0,0) after rotation = 0.707 0.707     0
```

3.3.2 变换矩阵

```cpp
    /*****************************************************************************
     *变换矩阵
     *****************************************************************************/
    /*定义，并用旋转矩阵、平移向量进行初始化*/
    //T为4*4。用于一个旋转3维向量
    Eigen::Isometry3d T=Eigen::Isometry3d::Identity();
    T.rotate ( rotation_vector );
    T.pretranslate ( Eigen::Vector3d ( 1,3,4 ) );
    cout << "Transform matrix = \n" << T.matrix() <<endl;
    /*使用变换矩阵*/
    Eigen::Vector3d v_transformed = T*v;
    cout<<"v tranformed = "<<v_transformed.transpose()<<endl<<endl;
```

```
Transform matrix =
 0.707 -0.707      0      1
 0.707  0.707      0      3
     0      0      1      4
     0      0      0      1
v tranformed = 1.71 3.71    4
```

3.3.3 欧拉角

```
    /*****************************************************************************
     *欧拉角;
     *****************************************************************************/
    Eigen::Vector3d euler_angles = rotation_matrix.eulerAngles ( 2,1,0 );
    cout<<"yaw pitch roll = "<<euler_angles.transpose()<<endl<<endl;
```

```
yaw pitch roll = 0.785    -0     0
```

3.4 四元数

```
    /*****************************************************************************
     *四元数
     *****************************************************************************/
    /*旋转矩阵转换为四元数*/
    Eigen::Quaterniond q = Eigen::Quaterniond ( rotation_vector );
    cout<<"quaternion = \n"<<q.coeffs() <<endl;
    /*旋转向量转换为四元数*/
    q = Eigen::Quaterniond ( rotation_matrix );
    cout<<"quaternion = \n"<<q.coeffs() <<endl;
    /*使用四元数*/
    v_rotated = q*v; //*经过了重载，原本：qvq-1
    cout<<"(1,0,0) after rotation = "<<v_rotated.transpose()<<endl;
```

```
quaternion =
    0
    0
0.383
0.924
quaternion =
    0
    0
0.383
0.924
(1,0,0) after rotation = 0.707 0.707     0
```

第4章 李群和李代数

旋转向量/矩阵、变换矩阵、欧拉角、四元数实现了对位姿的表示。在SLAM问题中，需要

对位姿进行估计和优化，因此通常构建优化问题进行求解。

在这过程涉及到求导问题，如果使用旋转矩阵，其本身带有约束（正交且行列式为1），将之作为优化变量会带来额外约束，从而使得优化求解变难。因此引出李群、李代数构建无约束优化问题，简化求解。

4.1 李群

群： 一种集合(A)加上一种运算$(.)$的代数结构：

$$G = (A; \cdot) \tag{4-1}$$

群满足以下条件：

名称	内容
封闭性	$\forall a_1, a_2 \in A, \quad a_1 \cdot a_2 \in A$
结合律	$\forall a_1, a_2, a_3 \in A, \quad (a_1 \cdot a_2) \cdot a_3 = a_1 \cdot (a_2 \cdot a_3)$
幺元	$\exists a_0 \in A, \quad \text{s.t.} \quad \forall a \in A, \quad a_0 \cdot a = a \cdot a_0 = a$
逆	$\forall a \in A, \quad \exists a_{-1} \in A, \quad \text{s.t.} \quad a \cdot a_{-1} = a_0$

矩阵中常见的群（$SO(n)$、$SE(n)$对乘法封闭，对加法不封闭）：

名称	内容
一般线性群$GL(n)$	$n \times n$的可逆矩阵，它们对矩阵乘法成群
特殊正交群$SO(n)$	也就是所谓的旋转矩阵群，其中$SO(2)$和$SO(3)$最为常见。
特殊欧式群$SE(n)$	也就是前面提到的n维欧氏变换，如$SE(2)$和$SE(3)$。

特殊正交群、特殊欧式群对乘法封闭，对加法不封闭。

李群： 具有连续性质的群。如整数群没有连续性质；特殊正交群、特殊欧式群是连续的，因为机器人/相机在空间中的运动时连续的，因此它们是李群。

4.2 李代数

4.2.1 李代数的引出

- 推导：

（1）旋转函数代表机器人/相机的旋转，随着时间连续变化，即为时间的函数。旋转矩阵是正交矩阵，有：

$$\boldsymbol{R}(t)\boldsymbol{R}(t)_{T}=\boldsymbol{I} \tag{4-2}$$

（2）等式两边对时间求导，可得出是一个反对称矩阵（2-3），又：定义向量对应的反对称矩阵，反对称矩阵对应的向量（2-4）。

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)_{T}+\boldsymbol{R}(t)\dot{\boldsymbol{R}}(t)_{T}=0 \tag{4-3}$$

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)_{T}=-\left(\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)_{T}\right)_{T} \tag{4-4}$$

$$\boldsymbol{a}_{\wedge}=\boldsymbol{A}=\begin{bmatrix}0 & -a_{3} & a_{2}\\ a_{3} & 0 & -a_{1}\\ -a_{2} & a_{1} & 0\end{bmatrix},\quad \boldsymbol{A}_{\vee}=\boldsymbol{a} \tag{4-5}$$

（3）因此可找到一个三维向量 $\phi(t)\in\mathbb{R}_{3}$ 与 $\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)_{T}$ 对应（2-5）。等式两边右乘 $\boldsymbol{R}(t)$，可得到每对旋转矩阵求一次导数，只需左乘一个 $\phi(t)_{\wedge}$ 矩阵即可（2-6）。

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)_{T}=\boldsymbol{\phi}(t)_{\wedge} \tag{4-6}$$

$$\dot{\boldsymbol{R}}(t)=\boldsymbol{\phi}(t)_{\wedge}\boldsymbol{R}(t)=\begin{bmatrix}0 & -\phi_{3} & \phi_{2}\\ \phi_{3} & 0 & -\phi_{1}\\ -\phi_{2} & \phi_{1} & 0\end{bmatrix}\boldsymbol{R}(t) \tag{4-7}$$

（4）令 $t_{0}=0$，且 $\phi\left(t_{0}\right)=\phi_{0}$，则有（2-7）；上式为微分方程，且有初值 $\boldsymbol{R}(0)=\boldsymbol{I}$，解之得（4-9）。

$$\dot{\boldsymbol{R}}(t)=\boldsymbol{\phi}\left(t_0\right)_{\wedge} \boldsymbol{R}(t)=\boldsymbol{\phi}_{\underset{0}{\wedge}} \boldsymbol{R}(t) \tag{4-8}$$

$$\boldsymbol{R}(t)=\exp\left(\phi_{\underset{0}{\wedge}} t\right) \tag{4-9}$$

- **结论：**

（1）ϕ：给定某时刻的$\boldsymbol{R}$，可求得一个ϕ，描述了$\boldsymbol{R}$在局部的导数关系。其中ϕ即是李代数。（4.2.2节）

（2）$\exp()$：矩阵指数。描述了李群和李代数之间的指数映射关系。（4.2.3节）

4.2.2 定义

- **李代数和李括号：**

李代数由一个集合$\mathbb{V}$，一个数域$\mathbb{F}$和一个二元运算$[,]$组成。如果它们满足以下几条性质，称$(\mathbb{V},\mathbb{F},[,])$为一个李代数。

名称	内容
封闭性	$\forall \boldsymbol{X},\boldsymbol{Y}\in\mathbb{V},[\boldsymbol{X},\boldsymbol{Y}]\in\mathbb{V}$
双线性	$\forall \boldsymbol{X},\boldsymbol{Y},\boldsymbol{Z}\in\mathbb{V},a,b\in\mathbb{F},\ [a\boldsymbol{X}+b\boldsymbol{Y},\boldsymbol{Z}]=a[\boldsymbol{X},\boldsymbol{Z}]+b[\boldsymbol{Y},\boldsymbol{Z}],\quad [\boldsymbol{Z},a\boldsymbol{X}+b\boldsymbol{Y}]=a[\boldsymbol{Z},\boldsymbol{X}]+b[\boldsymbol{Z},\boldsymbol{Y}]$
自反性	$\forall \boldsymbol{X}\in\mathbb{V},[\boldsymbol{X},\boldsymbol{X}]=\boldsymbol{0}$
雅可比矩阵	$\forall \boldsymbol{X},\boldsymbol{Y},\boldsymbol{Z}\in\mathbb{V},\ [\boldsymbol{X},[\boldsymbol{Y},\boldsymbol{Z}]]+[\boldsymbol{Z},[\boldsymbol{Y},\boldsymbol{X}]]+[\boldsymbol{Y},[\boldsymbol{Z},\boldsymbol{X}]]=0$

二元运算被称为李括号。

相比于群中的较为简单的二元运算，李括号表达了两个元素的差异。它不要求结合律，而要求元素和自己做李括号之后为零的性质。

- **李代数so(3)：**

$\mathfrak{so}(3)$是一个由三维向量组成的集合，每个向量对应一个反对称矩阵。根据2.1节推导可知，它可用于旋转矩阵的求导，李代数和李括号定义如下：

$$\mathfrak{so}(3)=\left\{\phi\in\mathbb{R}_3,\boldsymbol{\Phi}=\phi_\wedge=\begin{bmatrix}0&-\phi_3&\phi_2\\\phi_3&0&-\phi_1\\-\phi_2&\phi_1&0\end{bmatrix}\in\mathbb{R}_{3\times3}\right\}$$

$$[\phi_1,\phi_2]=(\boldsymbol{\Phi}_1\boldsymbol{\Phi}_2-\boldsymbol{\Phi}_2\boldsymbol{\Phi}_1)_\vee$$

- 李代数**se(3)**：

$\mathfrak{se}(3)$每个元素是一个六维向量记作$\boldsymbol{\xi}$：前三维是平移记作$\boldsymbol{\rho}$（这里的平移左乘一个系数矩阵才是变换矩阵中的真实平移$\boldsymbol{t}$，见2.2.2节），后三位是旋转记作$\boldsymbol{\phi}$（实质是$\mathfrak{so}(3)$的元素）。

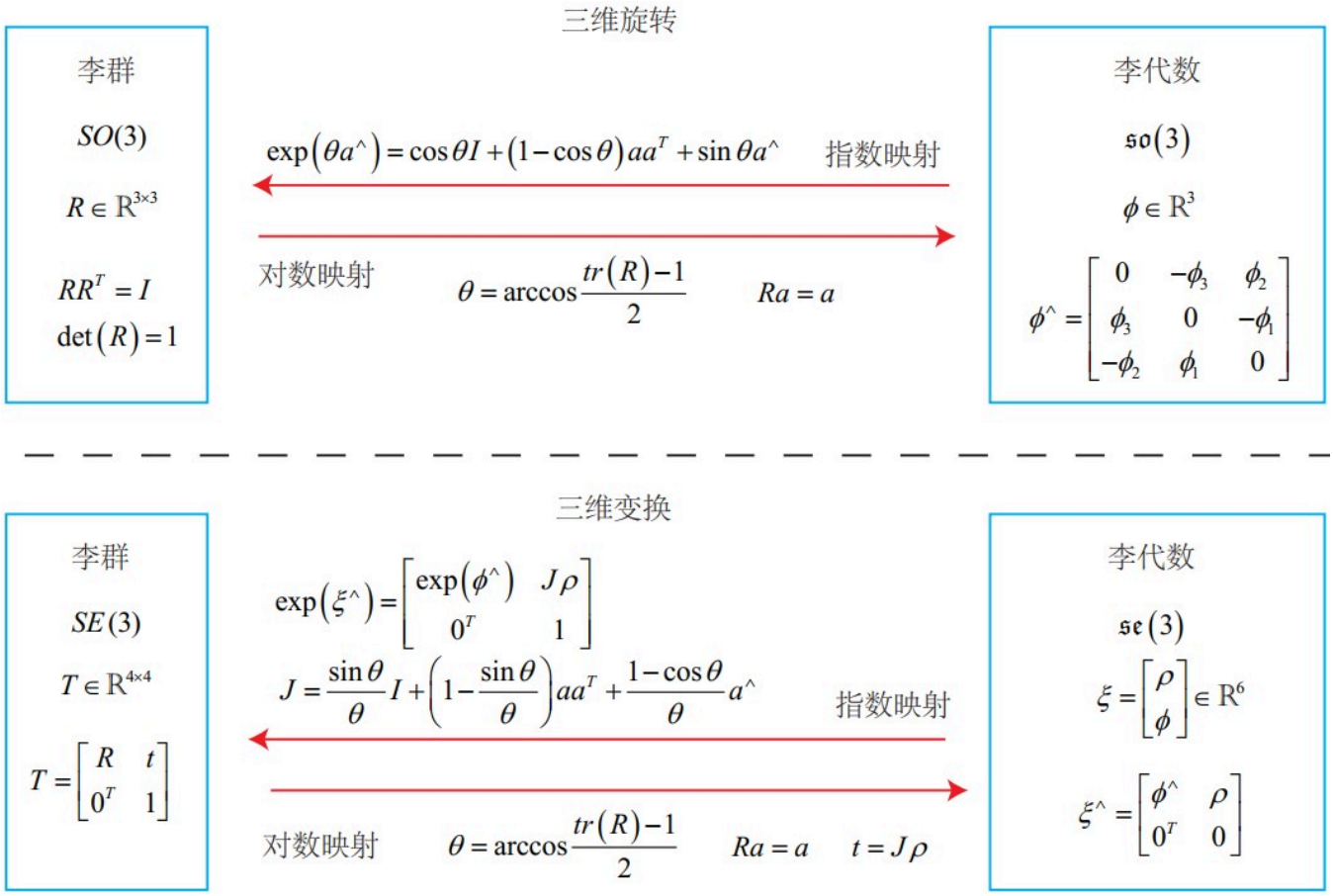
此处符号$_\wedge$和$_\vee$不代表反对称，而是被扩展成“六位向量”与“四维矩阵”的相互转换关系（为了保持和$\mathfrak{so}(3)$形式一致）。

李代数和李括号定义如下：

$$\mathfrak{se}(3)=\left\{\boldsymbol{\xi}=\begin{bmatrix}\boldsymbol{\rho}\\\boldsymbol{\phi}\end{bmatrix}\in\mathbb{R}_6,\boldsymbol{\rho}\in\mathbb{R}_3,\boldsymbol{\phi}\in\mathfrak{so}(3),\boldsymbol{\xi}_\wedge=\begin{bmatrix}\boldsymbol{\phi}_\wedge&\boldsymbol{\rho}\\\mathbf{0}_T&0\end{bmatrix}\in\mathbb{R}_{4\times4}\right\}$$

$$[\boldsymbol{\xi}_1,\boldsymbol{\xi}_2]=\left(\boldsymbol{\xi}_{1\wedge}\boldsymbol{\xi}_{2\wedge}-\boldsymbol{\xi}_{2\wedge}\boldsymbol{\xi}_{1\wedge}\right)_\vee$$

4.2.3 李群和李代数关系



4.2.3.1 SO(3)上的指数映射

$SO(3)$与$\mathfrak{so}(3)$的关系由指数映射给定：

$$\boldsymbol{R} = \exp\left(\boldsymbol{\phi}_\wedge\right)$$

- 推导：

（1）将指数映射写成泰勒展开式：

$$\exp\left(\boldsymbol{\phi}_\wedge\right) = \sum_{n=0}^{\infty} \frac{1}{n!} \left(\boldsymbol{\phi}_\wedge\right)_n$$

（2）因为$\boldsymbol{\phi}$是三维向量，将之表示为模长（θ）和方向（$\boldsymbol{a}$单位方向向量）的乘积，且具有以下性质：

$$\begin{cases}\phi = \theta \boldsymbol{a} \\ \boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} = \boldsymbol{a}\boldsymbol{a}_{T} - \boldsymbol{I} \\ \boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} = -\boldsymbol{a}_{\wedge}\end{cases}$$

（3）将上述表示带入指数映射：

$$\begin{aligned}\exp\left(\phi_{\wedge}\right) = \exp\left(\theta \boldsymbol{a}_{\wedge}\right) &= \sum_{n=0}^{\infty} \frac{1}{n!}\left(\theta \boldsymbol{a}_{\wedge}\right)_{n} \\ &= \boldsymbol{I} + \theta \boldsymbol{a}_{\wedge} + \frac{1}{2!}\theta_{2}\boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} + \frac{1}{3!}\theta_{3}\boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} + \frac{1}{4!}\theta_{4}\left(\boldsymbol{a}_{\wedge}\right)_{4} + \ldots \\ &= \boldsymbol{a}\boldsymbol{a}_{T} - \boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} + \theta \boldsymbol{a}_{\wedge} + \frac{1}{2!}\theta_{2}\boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} - \frac{1}{3!}\theta_{3}\boldsymbol{a}_{\wedge} - \frac{1}{4!}\theta_{4}\left(\boldsymbol{a}_{\wedge}\right)_{2} + \ldots \\ &= \boldsymbol{a}\boldsymbol{a}_{T} + \left(\theta - \frac{1}{3!}\theta_{3} + \frac{1}{5!}\theta_{5} - \ldots\right)\boldsymbol{a}_{\wedge} - \left(1 - \frac{1}{2!}\theta_{2} + \frac{1}{4!}\theta_{4} - \ldots\right)\boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} \\ &= \boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} + \boldsymbol{I} + \sin\theta \boldsymbol{a}_{\wedge} - \cos\theta \boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} \\ &= (1 - \cos\theta)\boldsymbol{a}_{\wedge}\boldsymbol{a}_{\wedge} + \boldsymbol{I} + \sin\theta \boldsymbol{a}_{\wedge} \\ &= \cos\theta \boldsymbol{I} + (1 - \cos\theta)\boldsymbol{a}\boldsymbol{a}_{T} + \sin\theta \boldsymbol{a}_{\wedge}\end{aligned}$$

- **结论：**

上式同罗德里格斯公式，即旋转向量与旋转矩阵的转换公式。因此如下结论：

（1）$\mathfrak{so}(3)$到$SO(3)$的指数映射：由罗德里格斯公式给出（2.6.2节），$\mathfrak{so}(3)$实际上就是旋转向量组成的空间。

（2）$SO(3)$到$\mathfrak{so}(3)$到的映射：由如下对数映射给出。但实际中常用2.6.2节所述方法，根据旋转矩阵计算旋转向量。

$$\phi = \ln(\boldsymbol{R})_{\vee} = \left(\sum_{n=0}^{\infty} \frac{(-1)_n}{n+1} (\boldsymbol{R} - \boldsymbol{I})_{n+1} \right)_{\vee}$$

4.2.3.2 SE(3)上的指数映射

$SE(3)$与$\mathfrak{se}(3)$的相互转换，可通过下述映射进行计算：

$$\begin{aligned} \exp\left(\boldsymbol{\xi}_{\wedge}\right) &= \begin{bmatrix} \sum_{n=0}^{\infty} \frac{1}{n!} \left(\boldsymbol{\phi}_{\wedge}\right)_n & \sum_{n=0}^{\infty} \frac{1}{(n+1)!} \left(\boldsymbol{\phi}_{\wedge}\right)_n \boldsymbol{\rho} \\ \boldsymbol{0}_T & 1 \end{bmatrix} \\ &\triangleq \begin{bmatrix} \boldsymbol{R} & \boldsymbol{J}\rho \\ \boldsymbol{0}_T & 1 \end{bmatrix} = \boldsymbol{T} \end{aligned}$$

$$J = \frac{\sin\theta}{\theta} \boldsymbol{I} + \left(1 - \frac{\sin\theta}{\theta}\right) \boldsymbol{a}\boldsymbol{a}_T + \frac{1 - \cos\theta}{\theta} \boldsymbol{a}_{\wedge}$$

4.3 · 李代数求导

4.3.1 BCH公式与近似形式

对于矩阵的指数函数，有BCH公式：

$$\ln(\exp(\boldsymbol{A})\exp(\boldsymbol{B})) = \boldsymbol{A} + \boldsymbol{B} + \frac{1}{2}[\boldsymbol{A}, \boldsymbol{B}] + \frac{1}{12}[\boldsymbol{A}, [\boldsymbol{A}, \boldsymbol{B}]] - \frac{1}{12}[\boldsymbol{B}, [\boldsymbol{A}, \boldsymbol{B}]] + \ldots$$

- **SO(3)上的近似形式**

取其近似形式，分为左乘近似和右乘近似：

$$\ln\left(\exp\left(\boldsymbol{\phi}_{1\wedge}\right)\exp\left(\boldsymbol{\phi}_{2\wedge}\right)\right)_{\vee} \approx \begin{cases} \boldsymbol{J}_l\left(\boldsymbol{\phi}_2\right)_{-1}\boldsymbol{\phi}_1 + \boldsymbol{\phi}_2 & \text{if } \boldsymbol{\phi}_1 \text{ is small,} \\ \boldsymbol{J}_r\left(\boldsymbol{\phi}_1\right)_{-1}\boldsymbol{\phi}_2 + \boldsymbol{\phi}_1 & \text{if } \boldsymbol{\phi}_2 \text{ is small.} \end{cases}$$

$$\boldsymbol{J}_l = \boldsymbol{J} = \frac{\sin\theta}{\theta}\boldsymbol{I} + \left(1 - \frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}_T + \frac{1-\cos\theta}{\theta}\boldsymbol{a}_\wedge$$

$$\boldsymbol{J}_r(\boldsymbol{\phi}) = \boldsymbol{J}_l(-\boldsymbol{\phi})$$

- **SE(3)上的近似形式：**

这里的雅可比矩阵形式复杂，是6×6的矩阵，实际计算中不会用到。

$$\exp\left(\Delta\boldsymbol{\xi}_\wedge\right)\exp\left(\boldsymbol{\xi}_\wedge\right) \approx \exp\left(\left(\mathcal{J}_{-1}\Delta\boldsymbol{\xi} + \boldsymbol{\xi}\right)_\wedge\right)$$
$$\exp\left(\boldsymbol{\xi}_\wedge\right)\exp\left(\Delta\boldsymbol{\xi}_\wedge\right) \approx \exp\left(\left(\mathcal{J}^l_{r\,-1}\Delta\boldsymbol{\xi} + \boldsymbol{\xi}\right)_\wedge\right)$$

4.3.2 李代数求导

4.3.2.1 李代数求导的实际意义

在 SLAM 中，我们要估计一个相机的位置和姿态，该位姿是由 $SO(3)$ 上的旋转矩阵或 $SE(3)$ 上的变换矩阵描述的。不妨设某个时刻小萝卜的位姿为 $\boldsymbol{T}$。它观察到了一个世界坐标位于 $\boldsymbol{p}$ 的点，产生了一个观测数据 $\boldsymbol{z}$。那么，由坐标变换关系知：

$$\boldsymbol{z} = \boldsymbol{T}\boldsymbol{p} + \boldsymbol{w}. \tag{4.37}$$

然而，由于观测噪声 $\boldsymbol{w}$ 的存在，$\boldsymbol{z}$ 往往不可能精确地满足 $\boldsymbol{z} = \boldsymbol{T}\boldsymbol{p}$ 的关系。所以，我们通常会计算理想的观测与实际数据的误差：

$$\boldsymbol{e} = \boldsymbol{z} - \boldsymbol{T}\boldsymbol{p}. \tag{4.38}$$

假设一共有 N 个这样的路标点和观测，于是就有 N 个上式。那么，对小萝卜的位姿估计，相当于是寻找一个最优的 $\boldsymbol{T}$，使得整体误差最小化：

$$\min_{\boldsymbol{T}} J(\boldsymbol{T}) = \sum_{i=1}^{N} \|\boldsymbol{z}_i - \boldsymbol{T}\boldsymbol{p}_i\|_2^2. \tag{4.39}$$

求解此问题，需要计算目标函数 J 关于变换矩阵 $\boldsymbol{T}$ 的导数。我们把具体的算法留到后面再讲。这里重点要说的是，**我们经常会构建与位姿有关的函数，然后讨论该函数关于位姿的导数，以调整当前的估计值。**

4.3.2.2 两种模型

第一章提到，SO 和 SE 没有加法，因此无法根据导数的定义对李群求导。可使用李代数解决该问题。

- **求导模型**

用李代数表示姿态，然后对根据李代数加法来对李代数求导（该模型计算中有较为复杂的雅可比矩阵，不常用）。

- **扰动模型**

对李群左乘或右乘微小扰动，然后对该扰动求导，称为左扰动和右扰动模型。

4.3.3 扰动模型

- **SO(3)上的求导：**

是对 $\boldsymbol{R}$ 进行一次扰动 $\Delta \boldsymbol{R}$ 。设左扰动 $\Delta \boldsymbol{R}$ 对应的李代数为 $\boldsymbol{\varphi}$ 。然后，对 $\boldsymbol{\varphi}$ 求导：

$$
\begin{aligned}
\frac{\partial(\boldsymbol{R}\boldsymbol{p})}{\partial \boldsymbol{\varphi}} &= \lim_{\boldsymbol{\varphi} \to 0} \frac{\exp\left(\boldsymbol{\varphi}_{\wedge}\right) \exp\left(\boldsymbol{\phi}_{\wedge}\right) \boldsymbol{p} - \exp\left(\boldsymbol{\phi}_{\wedge}\right) \boldsymbol{p}}{\boldsymbol{\varphi}} \\
&\approx \lim_{\boldsymbol{\varphi} \to 0} \frac{\left(1+\boldsymbol{\varphi}_{\wedge}\right) \exp\left(\boldsymbol{\phi}_{\wedge}\right) \boldsymbol{p} - \exp\left(\boldsymbol{\phi}_{\wedge}\right) \boldsymbol{p}}{\boldsymbol{\varphi}} \\
&= \lim_{\boldsymbol{\varphi} \to 0} \frac{\boldsymbol{\varphi}_{\wedge} \boldsymbol{R}\boldsymbol{p}}{\boldsymbol{\varphi}} = \lim_{\boldsymbol{\varphi} \to 0} \frac{-(\boldsymbol{R}\boldsymbol{p})_{\wedge} \boldsymbol{\varphi}}{\boldsymbol{\varphi}} = -(\boldsymbol{R}\boldsymbol{p})_{\wedge}
\end{aligned}
$$

- **SE(3)上的求导：**

$$
\begin{aligned}
\frac{\partial(\boldsymbol{T}p)}{\partial\delta\boldsymbol{\xi}} &= \lim_{\delta\boldsymbol{\xi}\to 0}\frac{\exp\left(\delta\boldsymbol{\xi}_{\wedge}\right)\exp\left(\boldsymbol{\xi}_{\wedge}\right)\boldsymbol{p}-\exp\left(\boldsymbol{\xi}_{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&\approx \lim_{\delta\boldsymbol{\xi}\to 0}\frac{\left(I+\delta\boldsymbol{\xi}_{\wedge}\right)\exp\left(\boldsymbol{\xi}_{\wedge}\right)\boldsymbol{p}-\exp\left(\boldsymbol{\xi}_{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to 0}\frac{\delta\boldsymbol{\xi}_{\wedge}\exp\left(\boldsymbol{\xi}_{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to 0}\frac{\begin{bmatrix}\delta\boldsymbol{\phi}_{\wedge} & \delta\boldsymbol{\rho}\\ \boldsymbol{0}_T^{\top} & 0\end{bmatrix}\begin{bmatrix}\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t}\\ 1\end{bmatrix}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to 0}\begin{bmatrix}\delta\boldsymbol{\phi}_{\wedge}(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t})+\delta\boldsymbol{\rho}\\ \boldsymbol{0}_T^{\top}\end{bmatrix}\frac{\square}{\delta\boldsymbol{\xi}} \\
&= \begin{bmatrix}\boldsymbol{I} & -(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t})_{\wedge}\\ \boldsymbol{0}_T^{\top} & \boldsymbol{0}_T^{\top}\end{bmatrix}\square\odot
\end{aligned}
$$

第5章 Sophus库 | 实现李代数运算

5.1 Sophus库

官网：https://github.com/strasdat/Sophus

安装：

```
git clone http://github.com/strasdat/Sophus.git
git checkout a621ff
mkdir build
cd build
cmake ..
make
```

5.2 SO(3)和SE(3)的运算

完整代码：https://github.com/deng-ke/slam-base/tree/master/SophusSOSE

5.2.1 构造李群SO(3)

```
/*******************************************************************************
*不同方式构造李群SO(3)
*******************************************************************************/
/*用旋转矩阵构造（沿Z轴转90度）*/
Eigen::Matrix3d R = Eigen::AngleAxisd(M_PI/2, Eigen::Vector3d(0,0,1)).toRotationMatrix
Sophus::SO3 SO3_R(R);
/*用旋转向量构造*/
Sophus::SO3 SO3_v( 0, 0, M_PI/2  );
/*用四元数构造*/
Eigen::Quaterniond q(R);
Sophus::SO3 SO3_q( q  );
cout << "SO(3) from matrix: " << SO3_R << endl;
cout << "SO(3) from vector: " << SO3_v << endl;
cout << "SO(3) from quaternion :" << SO3_q << endl;
```

```
SO(3) from matrix:      0      0 1.5708

SO(3) from vector:      0      0 1.5708

SO(3) from quaternion :     0      0 1.5708
```

5.2.2 李代数so(3)的运算

```
/*****************************************************************************
*李代数so(3)的运算
*****************************************************************************/
// 使用对数映射获得它的李代数
Eigen::Vector3d so3 = SO3_R.log();
cout<<"so3 = "<<so3.transpose()<<endl;
// hat 为向量到反对称矩阵
cout<<"so3 hat=\n"<<Sophus::SO3::hat(so3)<<endl;
// 相对的，vee为反对称到向量
cout<<"so3 hat vee= "<<Sophus::SO3::vee( Sophus::SO3::hat(so3)  ).transpose()<<endl; /
// 增量扰动模型的更新
Eigen::Vector3d update_so3(1e-4, 0, 0); //假设更新量为这么多
Sophus::SO3 SO3_updated = Sophus::SO3::exp(update_so3)*SO3_R;
cout<<"SO3 updated = "<<SO3_updated<<endl;
```

```
so3 =      0       0 1.5708
so3 hat=
      0 -1.5708       0
 1.5708       0      -0
     -0       0       0
so3 hat vee=       0       0 1.5708
SO3 updated =  7.85398e-05 -7.85398e-05       1.5708
```

5.2.3 构造李群SE(3)

```
/*****************************************************************************
*不同方式构造李群SE(3)
*****************************************************************************/
// 对SE(3)操作大同小异
Eigen::Vector3d t(1,0,0);           // 沿X轴平移1
Sophus::SE3 SE3_Rt(R, t);           // 从R,t构造SE(3)
Sophus::SE3 SE3_qt(q,t);            // 从q,t构造SE(3)
cout<<"SE3 from R,t= "<<endl<<SE3_Rt<<endl;
cout<<"SE3 from q,t= "<<endl<<SE3_qt<<endl;
```

```
SE3 from R,t=
     0      0 1.5708
1 0 0

SE3 from q,t=
     0      0 1.5708
1 0 0
```

5.2.4 李代数se(3)的运算

```
    /**********************************************************************************
    *李代数se(3)的运算
    **********************************************************************************/
    // 李代数se(3) 是一个六维向量，方便起见先typedef一下
    typedef Eigen::Matrix<double,6,1> Vector6d;
    Vector6d se3 = SE3_Rt.log();
    cout<<"se3 = "<<se3.transpose()<<endl;
    // 观察输出，会发现在Sophus中，se(3)的平移在前，旋转在后.
    // 同样的，有hat和vee两个算符
    cout<<"se3 hat = "<<endl<<Sophus::SE3::hat(se3)<<endl;
    cout<<"se3 hat vee = "<<Sophus::SE3::vee( Sophus::SE3::hat(se3)  ).transpose()<<endl;

    // 最后，演示一下更新
    Vector6d update_se3; //更新量
    update_se3.setZero();
    update_se3(0,0) = 1e-4d;
    Sophus::SE3 SE3_updated = Sophus::SE3::exp(update_se3)*SE3_Rt;
    cout<<"SE3 updated = "<<endl<<SE3_updated.matrix()<<endl;
```

```
se3 =  0.785398 -0.785398         0         0         0    1.5708
se3 hat =
        0   -1.5708         0  0.785398
   1.5708         0        -0 -0.785398
       -0         0         0         0
        0         0         0         0
se3 hat vee =  0.785398 -0.785398         0         0         0    1.5708
SE3 updated =
2.22045e-16          -1           0      1.0001
          1 2.22045e-16           0           0
          0           0           1           0
          0           0           0           1
```

第6章 相机模型

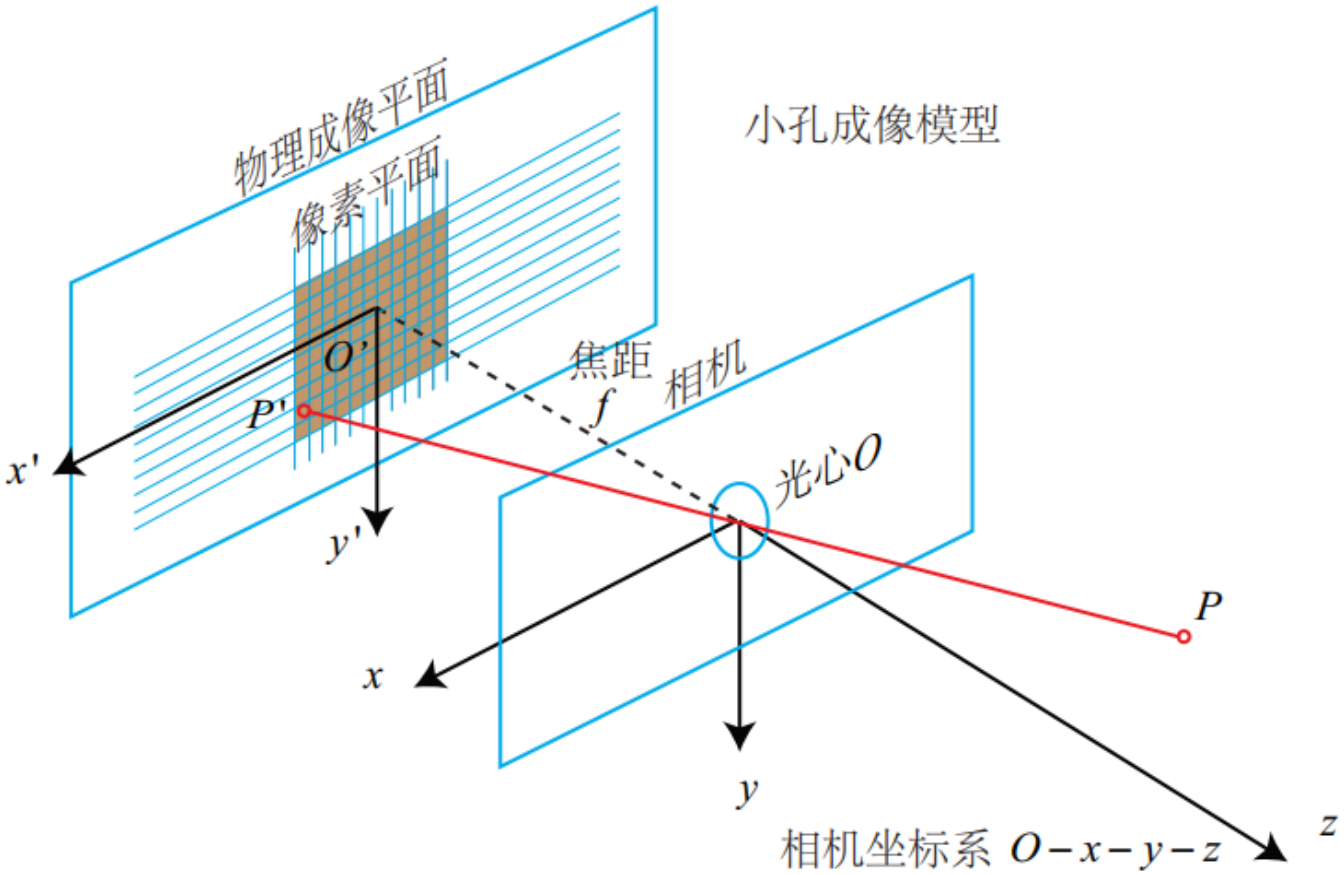
6.1 单目相机

6.1.1 针孔相机模型

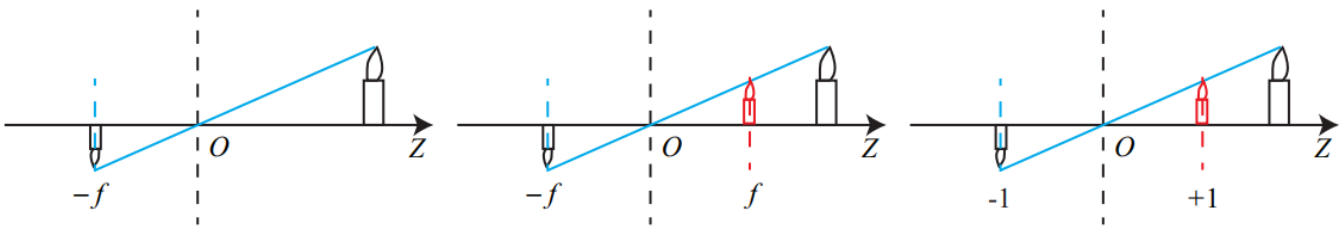
- 小孔成像：

相机坐标系下点$\boldsymbol{P}([X,Y,Z]_T)$，对应成像平面点$\boldsymbol{P}_{,}([X_{,},Y_{,},Z_{,}]_T)$，有如下关系。（为了简化模型，且考虑到相机会将倒像翻转，所以去掉负号关系）

$$\frac{Z}{f}=\frac{X}{X_{,}}=\frac{Y}{Y_{,}}\Rightarrow\begin{cases}X_{,}=f\frac{X}{Z}\\ Y_{,}=f\frac{Y}{Z}\end{cases}$$



便于数学运算，对成像平面进行处理：

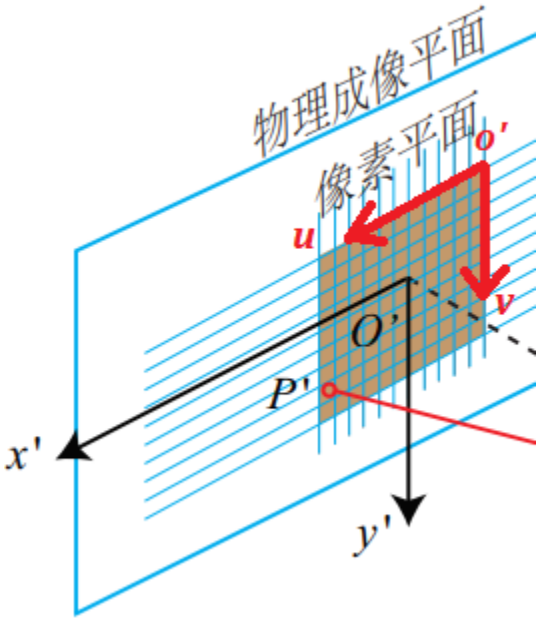


真实成像平面　　对称的成像平面　　归一化成像平面

- **像素坐标：**

像素坐标系与成像平面之间，相差了一个缩放和一个原点的平移。

$$\begin{cases} u = \alpha X_{,} + c_x \\ v = \beta Y_{,} + c_y \end{cases}$$



代入小孔成像的结果，其中f_x和f_y单位是像素：

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}$$

使用齐次坐标形式，写成矩阵形式。其中$(X/Z, Y/Z, 1)_T$为归一化坐标，它经过内参可得到像素坐标，因此可将像素坐标看作对归一化平面上的点进行量化测量的结果。

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{X}{Z} \\ \frac{Y}{Z} \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \triangleq \frac{1}{Z} \boldsymbol{KP}$$

$$Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \triangleq \boldsymbol{KP}$$

- **内参：**

$\boldsymbol{K}$，相机出厂后使用中，内参固定不变。使用时可能需要标定内参。

- **外参：**

代表相机轨迹的$\boldsymbol{R}$和$\boldsymbol{t}$。

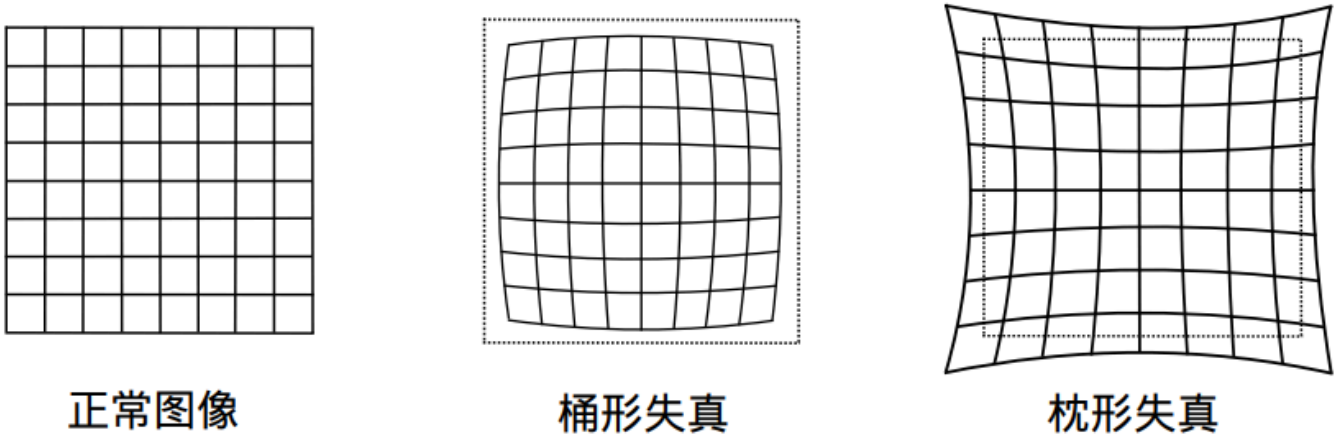
$$Z\boldsymbol{P}_{u,v} = Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \boldsymbol{K}\left(\boldsymbol{R}\boldsymbol{P}_w + \boldsymbol{t}\right) = \boldsymbol{K}\boldsymbol{T}\boldsymbol{P}_w$$

6.1.2 畸变

- **径向畸变**

透镜形状影响光线传播，引起畸变。主要有桶形失真、枕形失真。

径向畸变类型	含义
桶形畸变	由于图像放大率随着离光轴的距离增加而减小
枕形畸变	相反



径向畸变均是随着距离增加而增加，使用和距中心距离有关的二次及高次多项式进行纠正（极坐标）：

$$\begin{cases} \\ x_{corrected} = x(1 + k_1 r_2 + k_2 r_4 + k_3 r_6) \\ y_{corrected} = y(1 + k_1 r_2 + k_2 r_4 + k_3 r_6) \end{cases}$$

- 切向畸变

在机械组装过程中，透镜和成像平面不可能严格平行，引起畸变。

使用参数p_1, p_2纠正切向畸变

$$\begin{cases} \\ x_{corrected} = x + 2p_1 xy + p_2(r_2 + 2x_2) \\ y_{corrected} = y + p_1(r_2 + 2y_2) + 2p_2 xy \end{cases}$$

6.1.3 总结-成像过程

- 世界坐标系下一点P，世界坐标为$\boldsymbol{P}_w$
- 得到P点的相机坐标为：$\boldsymbol{P}_c = \boldsymbol{R}\boldsymbol{P}_w + \boldsymbol{t}$
- 得到P点的归一化相机坐标：投影到归一化平面$Z = 1$，$\boldsymbol{P}_c = [X/Z, Y/Z, 1]_T$
- 畸变纠正：对归一化平面的点进行径向畸变和切向畸变的纠正

$$\begin{cases} \\ x_{corrected} = x(1 + k_1 r_2 + k_2 r_4 + k_3 r_6) + 2p_1 xy + p_2(r_2 + 2x_2) \\ y_{corrected} = y(1 + k_1 r_2 + k_2 r_4 + k_3 r_6) + p_1(r_2 + 2y_2) + 2p_2 xy \end{cases}$$

- 得到像素坐标：$\boldsymbol{P}_{u,v} = \boldsymbol{K}\boldsymbol{P}_c$

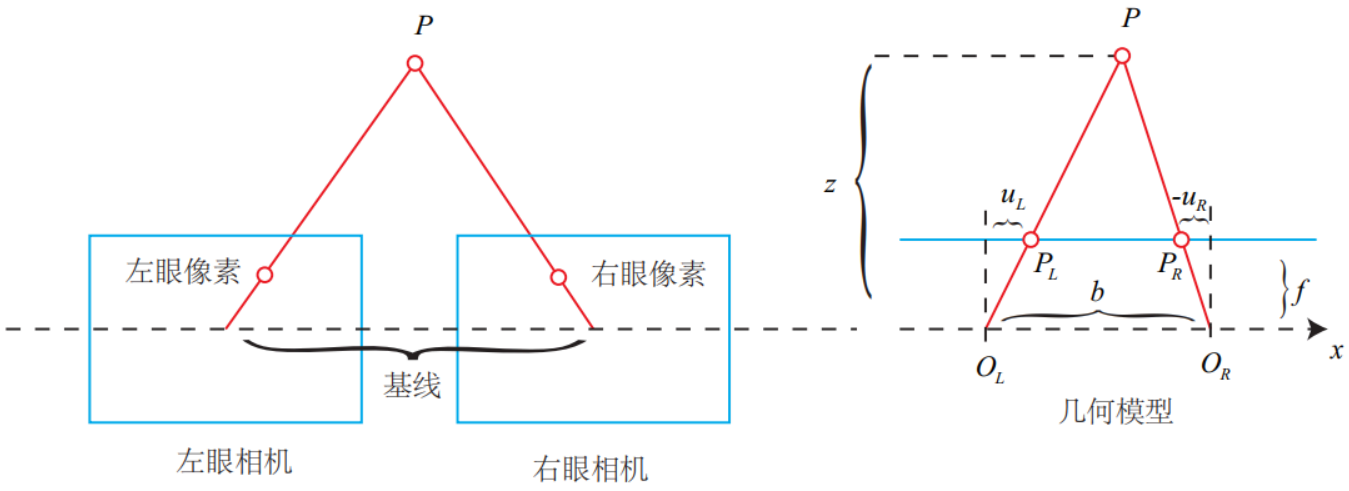
6.2 双目相机

根据三角形相似，得出深度的计算公式。

$$\frac{z - f}{z} = \frac{b - u_L + u_R}{b}$$

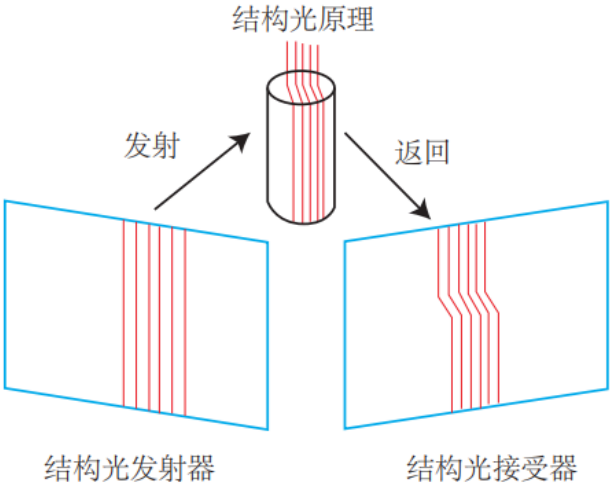
$$z = \frac{fb}{d}, \quad d = u_L - u_R$$

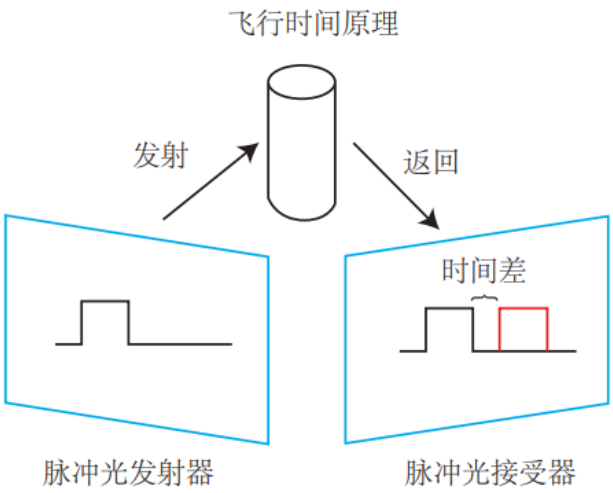
- 基线：左右光心间的距离，b。
- 视差：左右图的横坐标之差，d。
- 计算难度：左右眼图像的同步对应问题，导致d本身的计算较困难。因此计算量和精度成为问题。



6.3 RGB-D相机

- 红外结构光（Structured Light）：相机根据返回的结构光图案，计算物体离自身的距离。
- 飞行时间法（Time-of-flight, ToF）：相机向目标发射脉冲光，然后根据发送到返回之间的光束飞行时间，确定物体离自身的距离 。











第7章 PCL库 | 实现点云拼接

7.1 PCL库

官网：http://pointclouds.org/。

安装：（Ubuntu16.04）

```
sudo add-apt-repository ppa:v-launchpad-jochen-sprickerhof-de/pcl
sudo apt-get update
sudo apt-get install libpcl-dev
```

7.2 拼接点云

完整代码：

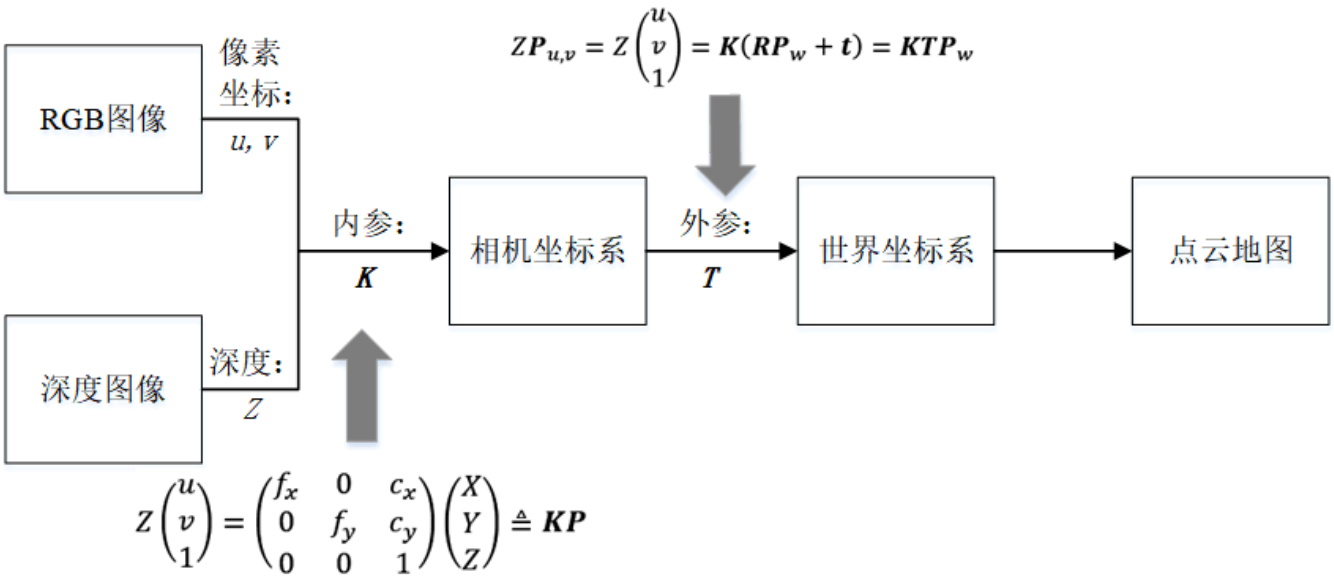
https://github.com/deng-ke/slam-base/tree/master/joinMap。

输入：

- color/：RGB图像
- depth/：深度图像
- pose.txt：每幅图对应的相机位姿（T_wc形式，与公式相反），格式为平移向量加四元数。

$$[x, y, z, q_x, q_y, q_z, q_w]$$

过程：



7.2.1 读入数据

读入RGB、深度图像，直接给定相机内参，读入外参。

```
/*读入RGB、深度图像，生成外参*/
for ( int i=0; i<5; i++ )
{
    boost::format fmt( "./%s/%d.%s" ); //图像文件格式
    colorImgs.push_back( cv::imread( (fmt%"color"%(i+1)%"png").str() ));
    depthImgs.push_back( cv::imread( (fmt%"depth"%(i+1)%"pgm").str(), -1 )); // 使用-1读取原

    double data[7] = {0};
    for ( auto& d:data )
        fin>>d;
    Eigen::Quaterniond q( data[6], data[3], data[4], data[5] );
    Eigen::Isometry3d T(q);
    T.pretranslate( Eigen::Vector3d( data[0], data[1], data[2] ));
    poses.push_back( T );
}
/*相机内参*/
double cx = 325.5;
double cy = 253.5;
double fx = 518.0;
double fy = 519.0;
```

7.2.2 转换到相机坐标

根据内参方程（逆向），将像素坐标转换为相机坐标。

```
point[2] = double(d)/depthScale;
point[0] = (u-cx)*point[2]/fx;
point[1] = (v-cy)*point[2]/fy;
```

7.2.3 转换到世界坐标

把相机坐标转换为世界坐标。

```
Eigen::Vector3d pointWorld = T*point;
```

7.2.4 拼接点云（地图）

```
PointT p ;
p.x = pointWorld[0];
p.y = pointWorld[1];
p.z = pointWorld[2];
p.b = color.data[ v*color.step+u*color.channels() ];
p.g = color.data[ v*color.step+u*color.channels()+1 ];
p.r = color.data[ v*color.step+u*color.channels()+2 ];
pointCloud->points.push_back( p );
```

7.2.5 结果



